

ML System Optimization – Programming Assignment (Assignment 2)

Group: 69

Title: Parallel Top-K Text Retrieval using TF-IDF + Cosine
Similarity with Sharded Top-K Merge

Facing Sheet

GitHub repository / notebook link:

https://github.com/mallinitc/As_I_Know/blob/main/BITS/MLSysOpt/Group69_Assignment2.ipynb

https://github.com/mallinitc/As_I_Know/blob/main/BITS/MLSysOpt/README.md

Team Contributions

ID Number	Name	Contribution
2024AC05653	D MALLIKARJUNA REDDY	100%
2024AD05166	AISHWARYA M	100%
2024AD05289	GEHI SAURABH	100%
2024AC05316	KONDALA UMAMAHESWARA RAO	100%
2024AC05997	SUBROTA BASU	100%

Table of Contents

1. Abstract
 2. Introduction
 3. P0 Problem Formulation
 4. P1 Design
 5. P1 (Revised) Implementation Details
 6. P2 Implementation
 7. P3 Experiments and Results
 8. Discussion
 9. Limitations
 10. Conclusion
- References

1. Abstract

We implement and evaluate a parallel version of a Top-K text retrieval workload on single-node Google Colab CPU. Tweets are embedded using TF-IDF vectors, cosine similarity is computed between each query and the full corpus, and the highest-scoring K documents are returned. We compare serial execution against a sharded parallel design with a Top-K merge step, and quantify runtime, speedup, and efficiency.

2. Introduction

Many real-world ML/IR pipelines spend significant time in similarity search (query vs. large corpus). This assignment focuses on the computational bottleneck in Top-K cosine similarity search over sparse TF-IDF vectors, and demonstrates when parallelization helps (and when overhead dominates) on a CPU-only Colab setting.

3. P0 Problem Formulation

Task.

Given a corpus of N documents and Q query documents, compute cosine similarity between each query vector and all corpus vectors, and return Top-K corpus indices per query.

Performance metrics.

Let $T(p)$ be the end-to-end runtime using p parallel workers ($p \in \{1, 2\}$ in our Colab CPU run). Speedup and efficiency are computed as: $S(p) = T(1)/T(p)$, $E(p) = S(p)/p$.

Platform assumption.

Single-node Google Colab runtime with 2 logical CPU cores; parallelism is implemented using multiprocessing (joblib).

4. P1 Design

We parallelize the workload by sharding the corpus matrix X_{corpus} into p disjoint blocks: $X_1, \dots, X_p$. For a given query vector q , each worker computes local similarities $s_i = \cos(q, X_i)$ and returns its local Top-K. A final merge step (TopKMerge) combines the p local Top-K lists into a global Top-K (K-way merge over $\leq p \cdot K$ candidates).

Key rationale: cosine similarity over sparse vectors is embarrassingly parallel across corpus shards; the merge is comparatively cheap ($O(p \cdot K \log(p \cdot K))$).

5. P1 (Revised) Implementation Details

Dataset.

Sentiment140 (1.6M tweets) loaded via Hugging Face; we sample $N = 50,000$ non-empty tweets as corpus and create Q queries ($Q = 50$ for initial checks, later increased to $Q = 400$ to amplify compute).

Vectorization.

TF-IDF vectorization using scikit-learn with stop-words removal; `max_features = 50,000` and `min_df = 2`. In our run, the resulting sparse matrix shape is (50,000, 16,235).

Parallel execution.

We use `joblib.Parallel` with p processes. Two variants are evaluated: (i) per-query parallelism (high overhead), and (ii) batched parallelism where each worker processes a batch of queries to amortize process scheduling and serialization.

6. P2 Implementation

The notebook implements: (a) a serial baseline that computes cosine similarity and Top-K per query, (b) a sharded worker function that returns local Top-K candidates, (c) `TopKMerge` that merges p local lists, and (d) an optimized batched path that reduces per-query parallel overhead.

7. P3 Experiments and Results

Environment summary.

CPU logical cores reported by Colab: 2.

Key timings from notebook outputs.

Setting	T(1) (s)	T(2) (s)	Notes
Q=50 (naive per-query parallel)	0.7679	8.5085	Parallel slower; overhead dominates
Q=50 (optimized batching)	0.7679	1.1771	Improved, but still slower than serial
Q=400 (median over 5 runs)	4.1086	2.1921	Speedup=1.874, Efficiency=0.937

For small workloads ($Q=50$), parallel execution is slower due to process start-up, task scheduling, and data movement. After batching and increasing workload ($Q=400$), we observe a clear improvement with $S(2) \approx 1.874$ and $E(2) \approx 0.937$ (median over repeated runs).

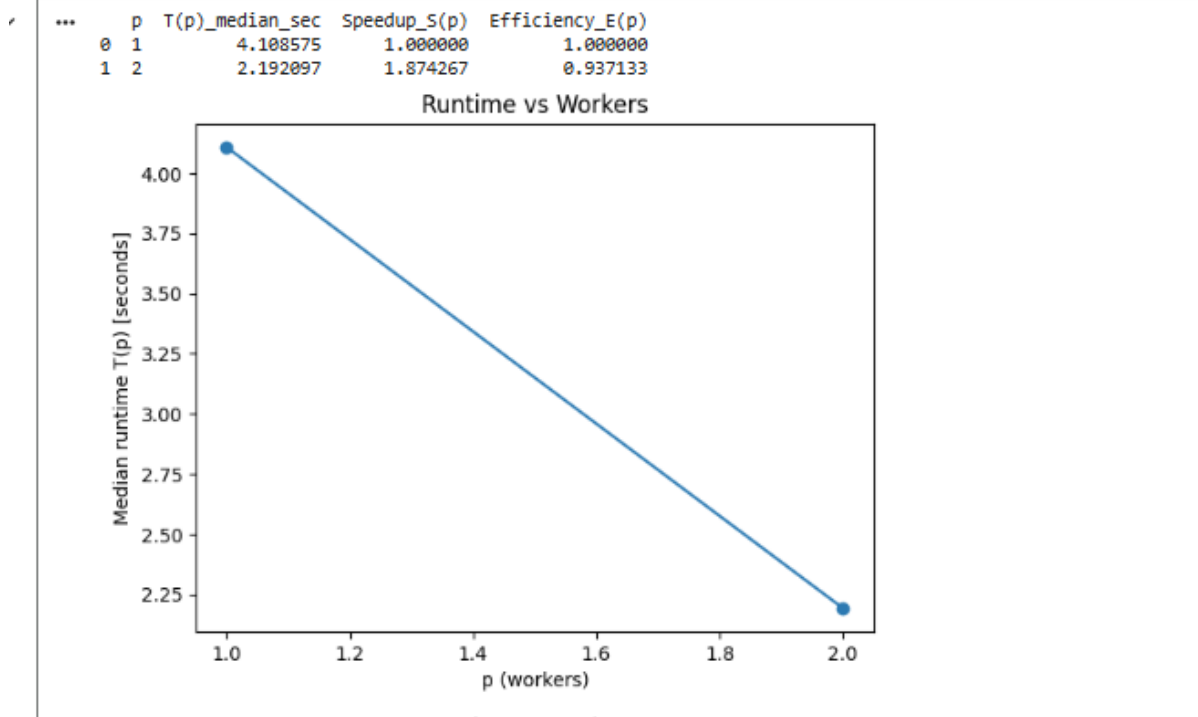


Figure 1: Runtime vs workers (median).

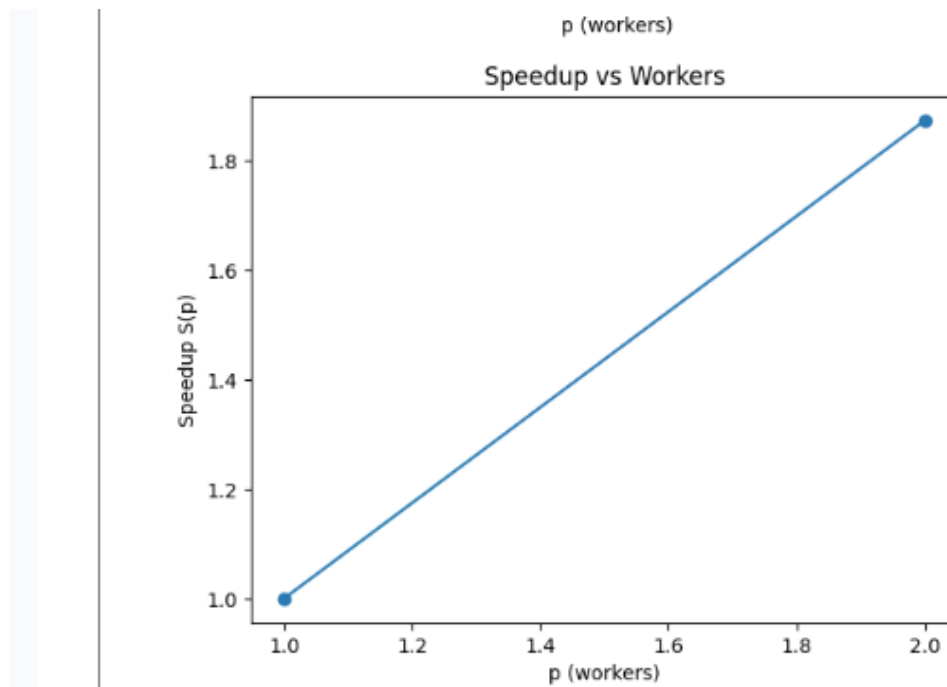


Figure 2: Speedup vs workers (median).

8. Discussion

The experiment highlights a common pitfall: naïvely parallelizing fine-grained tasks can reduce performance. In our case, per-query parallelism incurred high overhead (process scheduling + IPC/serialization of sparse slices). Batching increases compute per task and improves compute-to-overhead ratio, leading to meaningful speedup on $Q=400$.

Correctness checks show the Top-K sets match for most tested queries; occasional mismatches are expected when scores are tied or extremely close, where ordering can differ without affecting relevance.

9. Limitations

- Single-node Colab CPU; results may differ on multi-core machines or distributed clusters.
- We evaluate $p \in \{1, 2\}$ due to CPU availability; scaling trends beyond $p=2$ are not demonstrated.
- Dataset is sampled ($N=50k$) from Sentiment140; full-dataset indexing and memory constraints are out of scope.
- We focus on cosine similarity Top-K retrieval; other ANN methods (FAISS/HNSW) are not explored.

10. Conclusion

We implemented a sharded Top-K cosine similarity retrieval pipeline using TF-IDF vectors and evaluated serial vs parallel execution on Google Colab CPU. Results show that parallelization only becomes beneficial after reducing overhead (batching) and increasing workload. Using $p=2$ workers, the median speedup reaches ~ 1.87 with efficiency ~ 0.94 on $Q=400$, demonstrating measurable benefit while keeping outputs consistent.

References

- scikit-learn documentation: TfidfVectorizer and cosine_similarity.
- Sentiment140 dataset (Go et al., 2009) and Hugging Face dataset hosting.
- joblib documentation: Parallel and delayed (multiprocessing backend).